

GROUP 14: WACC PROJECT REPORT

The Final Product

The final version of our WACC compiler meets the functional requirements mentioned in the specification. It can successfully parse and generate assembly code for programs following the correct standards of the WACC programming language. If an invalid program is passed, then our compiler successfully identifies and produces meaningful error messages (messages including information such as line and column number, type of error, etc.). We used the provided example programs folder from the *wacc_examples* git repository and well as our own and using our integration tests to determine whether our compiler meets the specification.

We were provided with an online reference compiler, which we used as a tool to help understand advanced assembly code. Even though, there were some differences, the functionality was the same. We treated this as: 1) Same input, 2) Same output, 3) Same exit code. That said, we passed all the front-end tests, however we did not pass every single back-end test. But we insured that the same tests that passed in the back-end, pass after the extensions. When we looked at our generated assembly code, there were some design choices we made:

- 1) We removed unnecessary pushing and popping that was present as a base for each label in the reference compiler because whilst this didn't directly impact the result of the program it caused unnecessary bloat to the code which made it harder to understand.
- 2) We also added rigorous type safety to the arguments of instructions which prevented common errors such as trying to move a memory location to a memory location. Our general internal representation structures involving the various stages such as the IR blocks and the x86 Blocks were aided by this type of safety. This is because they prevented errors from developing during the translation between our more generalised function set and our very specific x86 blocks that catered to that architecture.
- 3) Our abstraction also focused on blocks and widgets, where we attempted to abstract all aspects of our code in the IR into more general instruction blocks which would give the most code flexibility for our future changes such as in the extensions. This abstraction would also make it simpler for us to change or alter how we implement certain WACC functionality.

If future development were to take place, we believe that we designed our codebase in such a way that this would be easy. We used several internal representations and templates, like the abstract syntax tree and symbol table, in our development. Thus, the action of adding new functionality to the compiler is not a difficult process. An example of this is our imports extension, where we were able to add new parsing rules, program AST objects, and code generation without much complexity. We originally planned to implement the option to compile to several processor architectures, however, due to use not fully implementing data types such as arrays and pairs, that we decided that it would be too much work, for too little credit to finish the back-end. The use of unit and integration testing helped us ensure we do not break any existing functionality in the compiler. We have made very clear using our documentation and comments of how our classes work.

Our code, especially after parsing is recursive and traverses the AST, performing IR/code generation as it sees each node. Many objects were recursive, such as expression, which are iterated through and then recursively traversed. This can be seen in our function `evalExpr()`. To sum up, we believe our compiler meets the functional specification of WACC, and even though we did not finish all the different types of data types, the code that we did implement was exceptionally reliable and efficient. Our decisions were carefully selecting with the future development of the project the primary objective. However, it was a shame that the back-end was not fully completed on time.

Project Management

Our approach to distributing the workload throughout the project was to develop parts of the compiler in parallel, making effective use of every single member of the team we had. Some tasks were done individually, while others were done in pairs on-campus. We use tools such as IntelliJ's Code with Me feature, to allow multiple people to code together, rather than all stare at one computer. The distribution of work depended on the feature and its dependence on other files. We typically started with easier tasks and made our way to longer tasks, allowing us to gradually build momentum and tackle more complex challenges with confidence and better knowledge of the codebase.

During the front-end, we first discussed how we were going to structure program. We talked about the AST, and we wrote as a team the types for each object to represent every possible type of term in a WACC program. This allowed us to then divide the group into

two. One, who dealt with parsing the plain text WACC file, and the other dealt with validating it for semantic checks. Doing it this way, allowed each pair to specialise in different fields. The first mastered the use of Parsley and the other with all the semantic rules of specification and perfected its error message.

The back-end was harder to divide, since a lot of features worked by themselves, but did not work together. For example, IO and If-statements were done by different people. When we had examples of programs with print statements inside of if-statements, it would not work, and we did not know whose fault was it. Printing of strings used Read-Only Data, which is assigned to each block of assembly code, and if statements split of code to different assembly blocks. Doing both together, did not work. So, we had both the person who did the IO and the if-statements to do pair programming together to figure out what the issue was and how to implement both features, correctly so that they worked with each other. This causes use to lose time but was necessary for development.

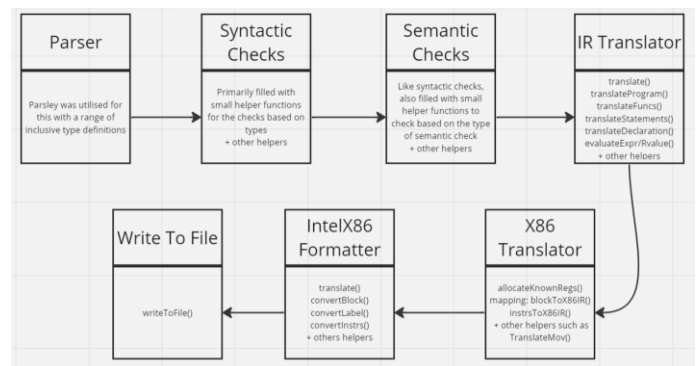
Our use of Git was crucial in our workflow. We established branches for each pair from master and constantly discusses what files we changed and how it would affect each of us. We constantly ran our integration tests, both locally and via our Gitlab pipeline, always checking for any errors from recent additions of new code. Over time, all branches were merged into master for LabTS testing or code submission.

A strength in our development was our IntelliJ's built in features to aid in development or debugging. We used the refactor feature to ensure that changes were made throughout the entire project. The debugger was extremely useful in figuring out incorrect values of variables that would have been difficult to realise, by eye.

A major weakness present through our development was our underestimation of how much code bases can change when we branched off. This caused significant issues in the merging, and we spent a long-time reimplanting feature that we already did in other branches.

Design Choices and Implementation Details

We decided to implement part of the WACC compiler using Scala's Parsley library for parser combinators. Scala's mixture of OOP and functional programming features allowed us to make full extent of pattern matching by abstract an intermediate assembly representation and an abstract syntax tree. Furthermore, we saw that there was significant support provided by Parsley (such as having a built-in lexer and syntax error messages) as well as abundant documentation online, as well as videos on Panopto detailing how the tool could be used in a project. This meant that we implemented the front-end faster than our peers with few concerns over basic functionality like lexing. Furthermore, this allowed us to concentrate on improving other areas of the front end, including semantic error messages in our semantic validator.



One feature of our front-end was the use of a global, flat symbol table. We opted for this approach due to its simplicity. Having one flat symbol table reduced the complexity of the symbol table and redundant variable names in the code, which made code more readable and easier to debug.

In the WACC back end, we split the code into two sections to enhance modularity, maintainability, and flexibility. We designed abstracted, non-language-specific instructions to make the code more general and enable the possibility of adding another backend in the future. By decoupling specific, assembly language specific instructions from the implementation, we achieved greater flexibility and extensibility. This abstraction would allow us to support multiple target architectures or even switch to a different assembly language backend without requiring significant modifications to the codebase if we ever needed to.

Secondly, we treated assembly code as a list of blocks, each encapsulating its specific state. This abstraction allowed us to manage code generation in a structured manner, simplifying the process of inserting or removing code segments as needed. By organizing the assembly code into blocks, we could isolate changes within specific sections, minimizing the risk of unintended side effects and facilitating code maintenance.

Also, we provided type safety for instructions, ensuring that only valid operations could be performed on operands of compatible types, such as attempting to perform a mov instruction from one memory location to another. This approach not only reduced the likelihood of runtime errors but also enhanced code readability and developer productivity by catching potential bugs at compile time. By enforcing type safety, we promoted robustness and correctness throughout the codebase.

Another abstraction we employed was having predefined widgets that are precompiled. By encapsulating these widgets separately, we avoided redundant compilation. Additionally, we dynamically added widgets as needed within the program rather than including them all at the end of the file. This approach optimized code generation by only including necessary components, reducing code bloat and improving performance.

Overall, these abstractions contributed to a more modular, maintainable, and adaptable compiler backend.

Beyond the Specification (Extensions)

Extension (Medium): Standard Library

In this medium extension task, the aim is to add the ability to import other written WACC files. However, unlike the easy task, we also needed to design a collection of useful functions that added extra functionality to the user's WACC program. To do this, we written both some functions in WACC as well as some in IR that were not possible in the standard WACC languages. These include a `Random` library, which provides random integer generator functions and the `Time` library, which provided functions to: get the current time, date and sleep a certain number of seconds. To implement these, the assembly code calls functions in the C library and returns the types, specified in the library documentation.

All imported functions that use other imported functions are present in the output file. To this, we recurse through the list of imports and each additional import (if there are any). Imports are then constructed into a directed graph (each node is a file, and each edge is an import relation) and imports are ordered such every function is defined before being called by another function (this is achieved by performing a topological sort on the graph).

The Assertion library contains functions like `Assertion.True(bool cond)` which is integrated into the IDE we have written and allows a way to create unit tests for WACC programs (mentioned in the next section).

Extension (Medium): WACC Integrated Development Environment

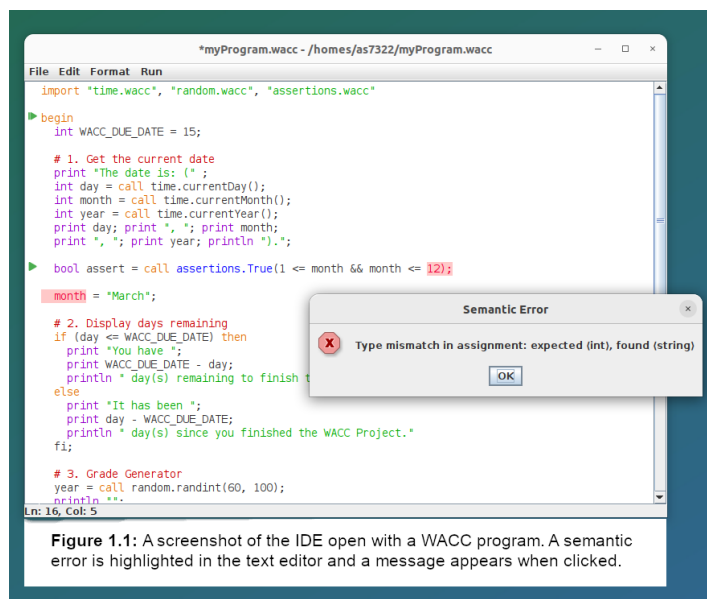


Figure 1.1: A screenshot of the IDE open with a WACC program. A semantic error is highlighted in the text editor and a message appears when clicked.

to assembly and then compiled with `gcc` and run.

As seen in Figure 1.1, the IDE has keyword highlighting as well as comment, string is updated after even time the text is modified. There is also automatic identification – when a new line is inserted, this line will have the same number of tabs as the previous line. When a newline is inserted, or the caret moves position, the IDE runs a syntax and semantic check. This is done by calling our front-end parser. If there are any errors, the code that is causing the error gets highlighted and the error message from the parser appears when this text is clicked (this can be seen in Figure 1.1).

The Assertion library is special as when a function from it is called, a run button appears on the line (as shown in Figure 1.1 and when clicked will compile and run the program, up-to and including that line which the button. If the test was successful (the program did not exit from -1 because of the assertion function, then a tick appears on the line. Otherwise, a red exclamation mark appears, telling the user that the assertion failed. Since it does not run the entire program, any run-time errors (for example) after the assert

In this medium extension task, we used to Java Swing library with Scala to create an IDE for the WACC language, incorporating many useful features to help programmers with the development of WACC programs.

Firstly, the WACC IDE has many useful text editor operations.

The first is creating a new file, where a default WACC program is automatically inserted, positioning the caret for the initial input within the begin-end block. Files can be opened, saved, or saved as per their existence or modification status. This is done by the user navigating through the operating system's file explorer.

Standard text manipulation functionalities like cut, copy, paste, and select all are integrated, which utilise the user's clipboard. Undo and redo functionalities are implemented by maintaining a stack of text states for restoration. Find and replace functionality is done through a dedicated window, enabling the identification and potential replacement of entered keywords. Formatting shortcuts are provided for indenting and commenting out blocks of code.

You can also run programs using the IDE. When "Run File" is clicked, a new terminal window opens, and the code is compiled

statement will not be run. A button also appears on the main begin-end block, which allows you to run the entire program instead and check for exit codes.

Extension (Medium) Optimisation – Parallel Compilation:

In our compiler implementation, we've incorporated a parallel compilation optimisation to enhance the efficiency of translating intermediate representation (IR) code as well as our x86 IR code into assembly language. This leverages concurrent programming techniques to distribute the compilation workload across multiple threads, thereby potentially reducing overall compilation time.

We divide the compilation process into smaller tasks, such as translating functions and translating the main program body. These tasks can be executed independently, making them suitable for concurrent execution.

We utilise Java's Future API and abstraction to represent asynchronous computations. This allows us to spawn tasks in parallel and asynchronously wait for their completion.

The `translateFuncsConcurrent()` method is responsible for concurrently translating functions present in the program. It spawns a separate thread for each function translation task using Scala's Future and collects the results.

The `translateProgram()` method handles the translation of the main program body. When concurrency is enabled, it launches the translation process for the main program body asynchronously, alongside the concurrent translation of functions.

We employ `futures` to combine the results of concurrent tasks. By using combinators like `map` and `flatMap`, we orchestrate the execution flow and synchronise the completion of tasks.

Parallel compilation reduces the overall compilation time by utilising multiple CPU cores efficiently. This results in faster feedback as we ensure that system resources are utilised optimally, leading to better scalability and responsiveness.

This optimisation significantly enhances the efficiency of the compilation process. By embracing concurrency, we strive to deliver a compiler that not only produces high-quality assembly output but also minimises compilation overhead, ultimately improving the overall developer experience.

Extension (Medium) Global Monomorphic Type Inference:

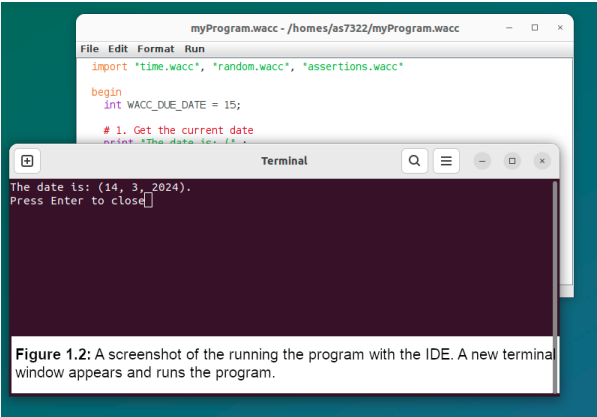
We enhanced the WACC compiler to incorporate locally inferred types, encompassing return types of functions and local variables. Our process began with designating the unknown types as `NoType` and then later attempting to assign their types. For declarations without types, we assign the variable type based on the expression they are assigned. Similarly, for functions without types, we assign their type based on their return expression.

Furthermore, we expanded it to deduce the types of function parameters, conducting an additional pass through each function's statements. Parameter types are determined by their usage; for instance, in an arithmetic binary operation, they are set to `IntType`. This modification allows functions to return parameters without specified types (`NoType`), triggering a semantic error through `semanticErrorOccured()`.

By implementing type inference, our WACC compiler facilitates simpler and more legible code. The utility of this addition is highlighted through the speed and efficiency of typing code, which helped us when we were developing tests for new WACC features in our extensions.

Extension (Medium) Full Pair Types:

With traditional WACC semantics, if we have the type `pair( $\tau$ ,  $\sigma$ )`, this denotes a pair whose first element has type τ and second element has type σ . However, when either of τ or σ within a pair are themselves a pair, they MUST have an erased type `pair`. For instance, `pair(int, pair)` is legal, but `pair(int, pair(int, bool))` is not. In this extension, we have made it so the latter is now legal. In other words, nested pairs no longer lose information. This was done by through `toFullPair()` in `checkStatements()` whenever a pair is declared or assigned. The `toFullPair()` function calls `pairElemToFullType()` on both pair elements using the value on the right-hand-side of the declaration (or assignment). Subsequently, we pattern match on all possible `rValues` and set the pair element type accordingly. Using a full example for understanding, here are the differences in the symbol table between traditional WACC semantics and the semantics following this extension when running the WACC file given in Figure 1.3:



```
begin
  pair(int, pair) p = newpair(11, null) ;
  pair(int, pair) q = newpair(4, p) ;
  pair(int, pair) r = newpair(2, q) ;
  pair(int, pair) s = newpair(1, r) ;

  pair(int, pair) x = s ;
  pair(int, pair) y = snd x ;

  println y
end
```

Figure 1.3

Symbol	Traditional WACC	Symbol Table with Full Pair Types
main-p	pair(int, pair)	pair(int, pair(AnyType, AnyType))
main-q	pair(int, pair)	pair(int, pair(int, pair(AnyType, AnyType)))
main-r	pair(int, pair)	pair(int, pair(int, pair(int, pair(AnyType, AnyType))))
main-s	pair(int, pair)	pair(int, pair(int, pair(int, pair(int, pair(AnyType, AnyType))))))

main-x	pair(int, pair)	pair(int, pair(int, pair(int, pair(int, pair(AnyType, AnyType))))))
main-y	pair(int, pair)	pair(int, pair(int, pair(int, pair(AnyType, AnyType))))

Note that `AnyType` is part of our compiler's internals and is never returned to the user explicitly. Instead, it is represented by the string "Unknown Type" in error messages.

Extension (Medium) Control-Flow Analysis:

In this medium task, control flow analysis was applied to improve the program's control flow structures within the abstract syntax tree, specifically targeting the optimisation of functions, while loops, and if statements, as well as improving the branching in the x86 translator to formatter. The extension uses an interpreter that tracks the state of the code, enabling the optimisation of conditions and function calls as they are encountered.

This extension adds two new stages to the compiler: the Control Flow AST (Abstract Syntax Tree) stage and the Control Flow Blocks stage.

The Control Flow AST stage focuses on optimising If and while loops as well as functions by computing conditions at compile time, which allows for replacing the loop with the correct branch directly or the function call with the side effects and return value. Additionally, While loops are optimised through unrolling to decrease condition recalculations and remove redundant jumps in later assembly, subject to a limit defined by the variable `MAX_UNROLLS` to prevent excessive code growth. For functions, the interpreter can compute the return value and the side effects (print statements). When a function is called, the control flow function code recursively calls itself with the given arguments since a function can be treated as a mini program with its arguments as the initial set of variables. The calls are substituted directly for computable print outputs or return values, and now unused functions are removed in the later Control Flow Blocks stage. Certain variables cannot be computed at compile time such as expressions including reads and thus a system of scala optional values is used to track the state of the program where an optional empty is given to a variable which cannot be known until runtime.

The Control Flow Blocks stage builds a control flow graph from the blocks produced by the x86 translator, which is then used to identify and eliminate unused functions or branches which removes dead code. Each label is treated as a vertex and each jump (condition + condition less) and call is treated as an edge. Depth first search starting from the main branch is used to find all the used code segments so that any non-accessed labels can then simply be removed. Every conditional jump is assumed to be possibly used depending on runtime variables given that compile-time calculatable conditions are optimised out during the AST stage of the control flow. Additionally, the graph structure has functions to detect any indirect branches/jumps. So, if branch A jumps to branch B, which then jumps to branch C, and branch B is not utilised elsewhere, this indirect jump is optimised by integrating branch B's code into branch A, thus directly linking A to C and eliminating the unnecessary jump through B that slows down the program.

However, this approach to optimising out branch B is applied only when branch B is not jumped to from multiple different branches. When branch B is accessed from multiple different points, integrating it into each jumping branch would significantly increase code duplication, leading to code bloat.